Project 1-2: Implementing DDL & Basic DML

Due: 2026/04/22 (Wed), 11:59 P.M.

1. Project Overview

이번 프로젝트의 목표는 프로젝트 1-1에서 구현한 SQL 파서 프로그램을 확장하여 DBMS(Database Management System)를 구현함으로써, 스키마를 저장하고 스키마에 접근할 수 있도록 하는 것이다.

프로젝트 1-2에서는 DBMS가 DDL(Data Definition Language) 구문과 DML(Data Manipulation Language) 구문을 처리할 수 있도록 한다.

1. **DDL 구문**: `CREATE TABLE`, `DROP TABLE`, `EXPLAIN/DESCRIBE/DESC`, `SHOW TABLES`, `RENAME TABLE`
2. **DML 구문**(일부): `INSERT INTO`, `SELECT`, `TRUNCATE TABLE`

1.1. LMDB (Lightning Memory-Mapped Database)

본 프로젝트부터는 LMDB 라이브러리를 사용한다. LMDB는 DBMS를 구현할 수 있도록 API를 제공하는 라이브러리로, 다음과 같은 특징을 가진다.

- Symas에서 공개한 오픈소스 라이브러리
- key-value pair를 byte array 형태로 저장하는 방식
- C++, Java, Python 등 다양한 언어의 API를 지원

LMDB는 relational system이 아니기 때문에, 스키마 설계 및 데이터 관리 방법을 자유롭게 고안해서 DBMS를 구현할 수 있다.

예)

- 하나의 DB파일에 하나의 스키마를 관리하는 방법 (One DB-One Schema)
- 하나의 DB파일에 복수의 스키마를 관리하는 방법 (One DB-Multi Schema)
- 스키마의 메타데이터를 별도의 DB파일에 저장, 관리하는 방법 (Metadata Schema) 등

Python LMDB API의 자세한 활용 방법은 6장의 Reference를 참고한다.

2. Requirements

이하의 조건들을 만족하도록 run.py 파일을 작성한다.

- 프로젝트 1-1에서 구현한 SQL 파서(grammar.lark, run.py)를 이용하여야 한다.
- 2.1장에 명시된 모든 구문들을 받아 올바르게 처리할 수 있어야 한다.
- 메시지 정의 문서(2026_1-2_Messages.docx)는 DBMS가 사용할 메시지의 종류와 그 내용을 정리한 문서다. 이를 참고하여 상황에 맞는 메시지를 프롬프트 뒤에 표시해야 한다.
  - ✓ 예시) `DB_2026-12345> Drop table has failed: no such table`
- 메시지 정의 문서에서 정의된 오류 유형 외의 유형이 추가로 필요하다고 판단될 경우 해당 유형의 이름과 메시지를 직접 정의하고 보고서에 명시하면 된다.
  - ✓ 본 문서에 정의되지 않은 유형은 평가 대상에 포함되지 않으므로 본 문서에 정의된 오류 유형을 우선 처리하는 것을 권장한다.
- 정답 외 불필요한 출력이 추가로 나올 경우 감점된다.
- 스키마를 파일(단일 파일 혹은 여러 개의 파일)에 저장하여야 한다.
  - ✓ LMDB 라이브러리를 이용하여 스키마를 저장 및 관리한다.
  - ✓ 프로그램을 종료한 후 다시 실행하더라도 저장된 DB 파일에 스키마가 남아 있어야 한다.
- Third-party libary을 통한 LMDB의 **SQL API 사용은 금지된다.**

2.1. SQL Queries

본 장에서는 각 구문에 대한 정의, 처리 방법과 입출력 예제를 제공한다.

2.1.1 CREATE TABLE

- Definition

```
create table table_name (
        column_name data_type [not null],
        …
        [primary key(column_name1, column_name2, …),]
        [foreign key(column_name3) references table_name1(column_name4),]
        [foreign key(column_name5) references table_name2(column_name6)]
        …
)
```

- 실행 예시

```
DB_2026-12345> create table account
(
        account_number int not null,
```

```
        branch_name char(15)
);
DB_2026-12345> 'account' table is created
DB_2026-12345>
```

- 입력한 쿼리가 올바르다면, 테이블 정보를 저장하고 `CreateTableSuccess(#tableName)`에 해당하는 메시지를 출력한다.
- 입력한 쿼리에 오류가 있다면, 오류에 대응되는 에러 메시지를 출력한다.
  - ✓ 컬럼의 이름이 중복될 경우, `DuplicateColumnDefError`에 해당하는 메시지 출력
  - ✓ Primary key 정의가 여러 번 입력된 경우, `DuplicatePrimaryKeyDefError`에 해당하는 메시지 출력
  - ✓ Foreign key의 타입과 foreign key가 참조하는 컬럼의 타입이 서로 다른 경우, `ReferenceTypeError`에 해당하는 메시지 출력
  - ✓ Foreign key가 primary key가 아닌 컬럼을 참조한다면, `ReferenceNonPrimaryKeyError`에 해당하는 메시지 출력
    - ➢ **(Foreign key가 composite primary key의 일부만을 reference하는 경우에도 `ReferenceNonPrimaryKeyError`에 해당하는 메시지 출력)**
  - ✓ Foreign key가 존재하지 않는 컬럼을 참조한다면, `ReferenceExistenceError`에 해당하는 메시지 출력
  - ✓ 존재하지 않는 컬럼을 primary key로 정의한 경우, `PrimaryKeyColumnDefError(#colName)`에 해당하는 메시지 출력
  - ✓ 존재하지 않는 컬럼을 foreign key로 정의한 경우에도 `ForeignKeyColumnDefError(#colName)`에 해당하는 메시지 출력
  - ✓ 이미 같은 이름의 테이블이 존재할 경우, `TableExistenceError`에 해당하는 메시지 출력
  - ✓ `char` 타입의 길이를 1보다 작게 지정한 경우, `CharLengthError`에 해당하는 메시지 출력
- 오류가 여러 개 동시에 발생하는 경우는 고려하지 않아도 된다.
- 테이블을 생성할 때에는 다음과 같은 가정을 따른다.
  - ✓ Primary key로 지정된 컬럼은 자동적으로 `not null`이 된다.
  - ✓ Primary key를 지정하지 않고도 테이블 생성이 가능하여야 한다.
  - ✓ Foreign key와 foreign key가 참조하는 컬럼의 타입은 서로 같아야 한다.
    - ➢ `char` 타입의 경우, 길이가 다르다면 서로 다른 타입으로 본다.

- ✓ `null` 값을 가질 수 있는 컬럼도 foreign key가 될 수 있다.
- ✓ 하나의 테이블은 여러 개의 foreign key를 가질 수 있다.
- ✓ `char` 타입의 길이는 0보다 커야 한다.
- ✓ 테이블 이름과 컬럼 이름은 대소문자를 구분하지 않는다. (case insensitive)
- ✓ `int`, `char` 이외의 데이터 타입 (`varchar`, `numeric`, `smallint`, `float` 등)은 구현하지 않아도 된다.
- ✓ Self-referencing foreign key(foreign key가 다른 테이블이 아닌 자기 자신 테이블을 참조하는 경우)는 고려하지 않아도 된다.
- ✓ Integrity constraint을 컬럼 정의와 함께 inline으로 작성하는 경우는 고려하지 않아도 된다.
  - ➢ 예. create table course (c_id varchar(8) primary key);

2.1.2 DROP TABLE

- Definition

```
drop table table_name;
```

- 실행 예시

```
DB_2026-12345> drop table account;
DB_2026-12345> 'account' table is dropped
DB_2026-12345>
```

- 입력한 쿼리가 올바르다면, 테이블 정보를 삭제하고 `DropSuccess(#tableName)`에 해당하는 메시지를 출력한다.
- 입력한 쿼리에 오류가 있다면, 오류에 대응되는 에러 메시지를 출력한다.
  - ✓ 테이블이 존재하지 않을 경우, `NoSuchTable(#commandName)`에 해당하는 메시지를 명령어와 함께 출력
    - ➢ E.g., `Drop table has failed: no such table`
  - ✓ 다른 테이블이 foreign key를 통해 현재 참조하고 있는 테이블을 삭제하려고 할 경우 (참조하던 테이블이 이미 삭제된 경우에는 정상적으로 drop이 가능), `DropReferencedTableError(#tableName)`에 해당하는 메시지 출력

2.1.3 EXPLAIN / DESCRIBE / DESC

- Definition

```
explain table_name;
describe table_name;
desc table_name;
```

- 실행 예시

```
DB_2026-12345> explain account;
-----------------------------------------------------------------
column_name         | type       | null        | key
account_number      | char(10)   | N           | PRI
branch_name         | char(15)   | N           | PRI/FOR
balance             | int        | Y           |
-----------------------------------------------------------------
3 rows in set
DB_2026-12345>
```

- 테이블 정보를 두 점선 사이에 출력한다.
  - ✓ 컬럼 이름, 타입, `null` 값 허용 여부, key 정보(primary key, foreign key)를 포함하여야 함.
  - ✓ 출력 형식은 위와 같이 각 컬럼 이름 (column_name, type, null, key), 그 다음으로 각 값들이 출력되어야 한다.
    - ➢ 필드 사이의 공백은 원하는 대로 정의하면 된다.
    - ➢ 또한, 데이터 출력 순서 관계없이 모든 데이터가 정상적으로 출력되기만 하면 된다.
- 테이블이 존재하지 않는다면 `NoSuchTable`에 해당하는 메시지를 명령어와 함께 출력한다.
- **describe, desc** 명령어에 대해서도 동일한 결과물을 출력할 수 있어야 한다.

2.1.4 SHOW TABLES

- Definition

```
show tables;
```

- 실행 예시

```
DB_2026-12345> show tables;
------------------------
branch
customer
loan
borrower
account
depositor
------------------------
6 rows in set
DB_2026-12345>
```

- DB에 존재하는 두 점선 사이에 모든 테이블의 이름을 출력하고, 점선 아래 테이블 개수를

출력한다.

- 아무 테이블도 존재하지 않는다면 두 점선과 '`0 rows in set`'을 출력한다.

2.1.5 INSERT

- Definition

```
insert into table_name [(col_name1, col_name2, …)] values (value1, value2, …);
```

- 실행 예시

```
DB_2026-12345> insert into account values(9732, 'Perryridge');
DB_2026-12345> The row is inserted
DB_2026-12345>
```

- 튜플(tuple)을 삽입할 때에는 다음과 같은 가정을 따른다.
  - ✓ `char` 컬럼 타입에 명시된 최대 길이보다 긴 문자열을 삽입하려 할 때는 에러를 발생시키지 않고 길이에 맞게 자른(truncate) 문자열을 삽입한다.
  - ✓ 테이블의 컬럼 이름은 중복되지 않는다.
  - ✓ 이번 프로젝트에서는 **'유효한 튜플'만이 삽입**되는 상황만 가정하고 구현하도록 한다. **'유효한 튜플'**이란 <u>테이블 정의에 위배되지 않는, 즉 아래 조건을 모두 충족하는 튜플</u>을 의미한다.
    - ➢ Column list를 명시하지 않은 경우, 입력된 value 개수가 테이블의 전체 컬럼 수와 일치
    - ➢ Column list를 명시한 경우, 입력된 value 개수가 명시된 컬럼 수와 일치하며, 명시되지 않은 컬럼은 null로 처리됨 (단, not null 컬럼은 반드시 column list에 포함되어야 함)
    - ➢ 각 컬럼의 자료형과 입력된 value들의 자료형이 모두 일치
    - ➢ not null에 해당하는 컬럼에는 null 값이 들어가지 않음
- 튜플 삽입에 성공한다면, 테이블에 값을 삽입하고 `InsertResult`에 해당하는 메시지를 출력한다.
- 입력한 쿼리에 오류가 있다면, 오류에 대응되는 에러 메시지를 출력한다.
  - ✓ 테이블이 존재하지 않을 경우, `NoSuchTable`에 해당하는 메시지 출력
  - ✓ <u>Key referential constraint 및 INSERT 구문 처리 과정에서 처리가 필요한 기타 오류들은 다음 프로젝트 (1-3)에서 구현할 예정이다</u>.

2.1.6 SELECT (이번 프로젝트에서는 predicate이 없는 'select *'만 구현)

- Definition

```
select * from table_name
```

- 실행 예시

```
DB_2026-12345> select * from account;
---------------------------------------
ACCOUNT_NUMBER | BRANCH_NAME | BALANCE
A-101          | Downtown    | 500
A-102          | Perryridge  | 400
A-201          | Brighton    | 900
A-215          | Mianus      | 700
A-217          | Brighton    | 750
A-222          | Redwood     | 700
A-305          | Round Hill  | 350
---------------------------------------
7 rows in set
DB_2026-12345>
```

- 입력한 쿼리가 올바르다면, 결과를 예시와 같은 형식으로 출력한다.
  - ✓ 점선, 필드 간의 공백은 원하는 대로 정의하면 된다.
  - ✓ 데이터 출력 순서 관계없이 모든 데이터가 정상적으로 출력되기만 하면 된다.
  - ✓ null 값을 가진 컬럼은 `null`로 출력한다.
  - ✓ '점선-데이터-점선'까지 출력한 다음에는 row 개수를 출력한다.
    - Row 개수가 1인 경우 `'1 row in set'`을 출력한다.
    - 아무 튜플도 존재하지 않는다면 두 점선과 `'0 rows in set'`을 출력한다.
- 입력한 쿼리에 오류가 있다면, 오류에 대응되는 에러 메시지를 출력한다.
  - ✓ `from`절에 있는 테이블이 존재하지 않는다면, `SelectTableExistenceError(#tabName)`에 해당하는 메시지를 출력
- 이번 프로젝트에서는 컬럼 선택, where절 등을 배제하고 한 테이블에 저장된 전체 데이터를 조회하는 기능만 구현한다.
- 이번 프로젝트에서는 FROM절에 두 개 이상의 테이블을 포함하는 경우는 고려하지 않는다.
- 본 기능이 구현되어야 **2.5 INSERT**장에 요구된 기능을 구현하고 동작 여부를 테스트할 수 있다. INSERT 구문과 마찬가지로 SELECT 구문은 다음 프로젝트 (1-3)에서 완성할 예정이다.

2.1.7 RENAME (이번 프로젝트에서는 한번 input당 하나의 rename 만 구현)

- Definition

```
rename table table_name to new_table_name
```

- 실행 예시

```
DB_2026-12345> rename table account to all_accounts;
DB_2026-12345> 'all_accounts' is renamed
DB_2026-12345>
```

- 입력한 쿼리가 올바르다면, 테이블 이름을 바꾸고 `RenameSuccess(#tableName)` 에 해당하는 메시지를 출력한다.
- 입력한 쿼리에 오류가 있다면, 오류에 대응되는 에러 메시지를 출력한다.
  - ✓ 테이블이 존재하지 않을 경우, `NoSuchTable(#commandName)`에 해당하는 메시지를 명령어와 함께 출력
    - ➢ E.g., `Rename table has failed: no such table`
  - ✓ 변경하려는 이름(new_table_name)의 테이블이 이미 존재할 경우, `RenameAlreadyExistError`에 해당하는 메시지 출력

2.1.8 TRUNCATE

- Definition

```
truncate table table_name
```

- 실행 예시

```
DB_2026-12345> truncate table all_accounts
DB_2026-12345> 'all_accounts' is truncated
DB_2026-12345>
```

- 입력한 쿼리가 올바르다면, 테이블을 비우고 `TruncateSuccess(#tableName)` 에 해당하는 메시지를 출력한다.
- 입력한 쿼리에 오류가 있다면, 오류에 대응되는 에러 메시지를 출력한다.
  - ✓ 테이블이 존재하지 않을 경우, `NoSuchTable(#commandName)`에 해당하는 메시지를 명령어와 함께 출력
    - ➢ E.g., `Truncate table has failed: no such table`
  - ✓ 다른 테이블이 해당 테이블을 foreign key로 참조하고 있는 경우, `TruncateReferencedTableError(#tableName)`에 해당하는 메시지 출력

3. 개발 환경

- Python 3.10 ~ 3.12
- Lark API
- LMDB API

4. 제출

다음 파일들을 PRJ1-2_학번.zip(예: PRJ1-2_2026-12345.zip)으로 압축하여 제출한다.

1. grammar.lark
2. run.py
   - ✓ 프로젝트의 최상위 디렉토리에 위치해야 한다.
   - ✓ 추가적인 소스코드 파일 및 서브 디렉토리를 함께 제출해도 된다. 단, python run.py 로 프로그램이 구동될 수 있도록 해야 한다.
   - ✓ 적절한 주석을 포함해야 한다.
3. 리포트
   - ✓ 프로젝트의 최상위 디렉토리에 위치해야 한다.
   - ✓ 반드시 pdf 형식이어야 한다.
   - ✓ 파일명은 PRJ1-2_학번.pdf (예: PRJ1-2_2026-12345.pdf)으로 한다.
   - ✓ 2장 이내로 작성한다(1장을 권장).
   - ✓ 반드시 포함되어야 하는 내용
     - ➢ 핵심 모듈과 알고리즘에 대한 설명
     - ➢ 구현한 내용에 대한 간략한 설명
     - ➢ (제시된 요구사항 중 구현하지 못한 부분이 있다면) 구현하지 못한 내용
     - ➢ 프로젝트를 하면서 느낀 점 및 기타사항
   - ✓ 추가로 포함할 수 있는 내용
     - ➢ 본 문서에 정의된 오류 유형 외 추가로 정의한 오류 유형

5. 성적 관련 사항

- 제출 기한 이후 24시간 이내 제출시 10% 감점
- 제출 기한 이후 24시간 이후 48시간 이내 제출시 20% 감점
- 제출 기한 48시간 이후에는 점수 없음
- 부정 행위는 0점 처리
  - ✓ 다른 사람의 코드를 참조하는 행위
  - ✓ 이전에 수강한 사람의 코드를 참조하는 행위
  - ✓ 제출한 소스코드에 대해 표절 방지 프로그램을 돌릴 예정
- 본 문서에 명시되어 있는 출력 양식을 지키지 않았거나 주석이 없는 경우 감점

6. References

- **LMDB**
  - ✓ https://www.symas.com/mdb
- **Python LMDB API Resources**
  - ✓ https://lmdb.readthedocs.io/en/latest